1. Medical Diagnosis using Knowledge Base

```
# Educational rule-based example only; not for real diagnosis.

facts = {
    "fever",
    "cough",
    "sore_throat"
}

# Rules are represented as (conditions, conclusion).
rules = [
    ({"fever", "cough"}, "possible_flu"),
    ({"fever", "sore_throat"}, "possible_throat_infection"),
    ({"possible_flu", "sore_throat"}, "recommend_medical_review")
]

def forward_chaining(facts, rules):
    known = set(facts)
    changed = True

    while changed:
        changed = False

        for conditions, conclusion in rules:
            if conditions.issubset(known) and conclusion not in known:
                known.add(conclusion)
                print("Derived:", conclusion)
                changed = True

    return known

def backward_chaining(goal, facts, rules, visited=None):
    if visited is None:
        visited = set()

    if goal in facts:
        return True

    if goal in visited:
        return False

    visited.add(goal)

    for conditions, conclusion in rules:
        if conclusion == goal:
            if all(
                backward_chaining(condition, facts, rules, visited.copy())
                for condition in conditions
            ):
                return True

    return False

print("Initial facts:", sorted(facts))

print("\nForward chaining:")
known_facts = forward_chaining(facts, rules)
print("Knowledge base:", sorted(known_facts))

print("\nBackward chaining:")
goal = "recommend_medical_review"
if backward_chaining(goal, facts, rules):
    print("Goal proved:", goal)
else:
    print("Goal cannot be proved:", goal)
```

2. Autonomous Vehicle using Knowledge Base

```
# Rule-based decisions for a simplified autonomous vehicle.

facts = {
    "obstacle_ahead",
    "speed_high",
    "traffic_light_red",
    "pedestrian_detected"
}

rules = [
    ({"obstacle_ahead"}, "brake"),
    ({"traffic_light_red"}, "stop"),
    ({"pedestrian_detected"}, "stop"),
    ({"speed_high", "obstacle_ahead"}, "emergency_braking"),
    ({"traffic_light_red", "pedestrian_detected"}, "remain_stopped")
]

def forward_chaining(facts, rules):
    known = set(facts)
```

```
    changed = True

    while changed:
        changed = False

        for conditions, action in rules:
            if conditions.issubset(known) and action not in known:
                known.add(action)
                print("New decision:", action)
                changed = True

    return known

def backward_chaining(goal, facts, rules, visited=None):
    if visited is None:
        visited = set()

    if goal in facts:
        return True

    if goal in visited:
        return False

    visited.add(goal)

    for conditions, conclusion in rules:
        if conclusion == goal:
            if all(
                backward_chaining(c, facts, rules, visited.copy())
                for c in conditions
            ):
                return True

    return False

print("Sensor facts:", sorted(facts))

print("\nForward chaining decisions:")
decisions = forward_chaining(facts, rules)
print("All known facts and decisions:", sorted(decisions))

print("\nBackward chaining:")
goal = "emergency_braking"
if backward_chaining(goal, facts, rules):
    print("Required decision proved:", goal)
else:
    print("Required decision not proved:", goal)
```

3. Industrial Manufacturing using Knowledge Base

```
# Rule-based monitoring in an industrial manufacturing unit.

facts = {
    "machine_temperature_high",
    "vibration_high",
    "production_rate_low",
    "maintenance_due"
}

rules = [
    ({"machine_temperature_high"}, "inspect_cooling_system"),
    ({"vibration_high"}, "inspect_machine_alignment"),
    ({"production_rate_low", "vibration_high"}, "possible_machine_fault"),
    ({"maintenance_due"}, "schedule_maintenance"),
    ({"machine_temperature_high", "vibration_high"},
     "stop_machine_for_inspection"),
    ({"possible_machine_fault", "maintenance_due"},
     "priority_maintenance_required")
]

def forward_chaining(facts, rules):
    known = set(facts)
    changed = True

    while changed:
        changed = False

        for conditions, conclusion in rules:
            if conditions.issubset(known) and conclusion not in known:
                known.add(conclusion)
                print("Derived:", conclusion)
                changed = True

    return known

def backward_chaining(goal, facts, rules, visited=None):
    if visited is None:
        visited = set()
```

```
    if goal in facts:
        return True

    if goal in visited:
        return False

    visited.add(goal)

    for conditions, conclusion in rules:
        if conclusion == goal:
            if all(
                backward_chaining(c, facts, rules, visited.copy())
                for c in conditions
            ):
                return True

    return False

print("Production facts:", sorted(facts))

print("\nForward chaining:")
known_facts = forward_chaining(facts, rules)
print("Knowledge base:", sorted(known_facts))

print("\nBackward chaining:")
goal = "priority_maintenance_required"
if backward_chaining(goal, facts, rules):
    print("Goal proved:", goal)
else:
    print("Goal cannot be proved:", goal)
```